

3. Write the python program for Water Jug Problem

Aim

To develop a Python program to solve the **Water Jug Problem** using the **Breadth-First Search (BFS) Algorithm** by accepting the capacities of two jugs and the target amount from the user.

Algorithm

1. Start the program.
 2. Read the capacities of Jug 1, Jug 2, and the target amount.
 3. Initialize the initial state as (0, 0).
 4. Insert the initial state into a queue.
 5. Repeat until the queue becomes empty:
 - Remove the front state.
 - If the target amount is reached, display the solution path and stop.
 - Generate all possible next states:
 - Fill Jug 1.
 - Fill Jug 2.
 - Empty Jug 1.
 - Empty Jug 2.
 - Pour water from Jug 1 to Jug 2.
 - Pour water from Jug 2 to Jug 1.
 - Add all valid and unvisited states to the queue.
 6. If no solution is found, display "No Solution Exists."
 7. Stop.
-

Program

```
from collections import deque

# Function to solve Water Jug Problem using BFS
def water_jug_bfs(jug1, jug2, target):
    visited = set()
    queue = deque()
    # Initial state
```

```

queue.append((0, 0, []))
while queue:
    x, y, path = queue.popleft()
    if (x, y) in visited:
        continue
    visited.add((x, y))
    path = path + [(x, y)]
    # Goal condition
    if x == target or y == target:
        return path
    # Possible operations
    next_states = [
        (jug1, y),          # Fill Jug1
        (x, jug2),          # Fill Jug2
        (0, y),             # Empty Jug1
        (x, 0),             # Empty Jug2
    ]
    # Pour Jug1 -> Jug2
    transfer = min(x, jug2 - y)
    next_states.append((x - transfer, y + transfer))
    # Pour Jug2 -> Jug1
    transfer = min(y, jug1 - x)
    next_states.append((x + transfer, y - transfer))
    for state in next_states:
        if state not in visited:
            queue.append((state[0], state[1], path))
return None

# Main Program
jug1 = int(input("Enter capacity of Jug 1: "))
jug2 = int(input("Enter capacity of Jug 2: "))
target = int(input("Enter target amount: "))
solution = water_jug_bfs(jug1, jug2, target)
if solution:

```

```
        print("\nSolution Path:")
        for step in solution:
            print(step)
else:
    print("\nNo Solution Exists.")
```

Input

```
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target amount: 2
```

Output

```
Solution Path:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
>>> |
```

Result

The Python program was successfully implemented to solve the **Water Jug Problem** using the **Breadth-First Search (BFS) Algorithm**. The program accepts the capacities of the two jugs and the target amount from the user, finds a valid sequence of operations, and displays the intermediate states required to obtain the target amount of water. This demonstrates the use of BFS in solving state-space search problems in Artificial Intelligence.